机器人模型及数据统一管理方案

初版资源管理方案 · 修订 0.3 · 2026年11月5日

方案概要

建立独立的 GitHub 仓库 **rocos-sia/rocos-resources**，集中保存机械臂型号、每台设备的配置与标定数据、工具模型以及启动所需资料。初版围绕“统一存放、方便下载、直接使用、持续维护”组织，采用普通目录和文件管理。

所有运行资料放在仓库的 config/ 目录。其中 app.yaml、ecm.yaml、igh.yaml 等根目录入口文件是配置模板。使用前通过 rocos-cfg 选择型号、设备和主站，完成实际参数配置，再将导出的入口文件更新到目标 config 目录，例如 /opt/rocos/config。rocos-app、rocos-ecm、rocos-igh 等共用更新后的这一份配置目录。

1 仓库保存哪些资料

目录或文件	保存内容	组织方式
config/robots/	机械臂型号配置、URDF、mesh、名义硬件参数、ENI、主站相关配置及仿真模型	按型号建立目录，供同型号设备引用
config/devices/	每台机器人的设备配置、安装信息、零点、运动学和动力学标定、负载与 TCP 数据、维护记录	按“名称_序列号”建立独立设备档案
config/tools/	工具配置、工具模型与 mesh 等可复用资料	按工具名称建立目录；设备记录实际安装关系
config/ 下的入口模板	App、ECM、IgH 等程序的启动配置模板及用户差量模板	使用前由 rocos-cfg 按实际工程配置更新或导出，再引用所选 robots、devices、tools 资料
config/scripts/	各程序的启动脚本和可复用示例脚本	按用途命名，在 README 中说明使用方式

rocos-resources 维护共用资源和入口模板；rocos-app、rocos-ecm、rocos-igh 等仓库维护各自程序代码。rocos-cfg 负责基于模板配置实际工程，选择型号、设备、工具和主站，并更新或导出相应配置。实际使用时，将导出的入口文件及相关配置放回同一套 config 目录结构中。

2 建议目录结构

首版尽量沿用现有文件名与相对路径关系，先将资料集中起来。以下目录展示建议组织方式；具体文件只在有实际资料时添加。

Code block

```
rocos-resources/
  README.md
  config/
    app.yaml                        # App 入口模板 由 rocos-cfg 配置更新
    ecm.yaml                        # ECM 入口模板 由 rocos-cfg 配置更新
    igh.yaml                        # IgH 入口模板 由 rocos-cfg 配置更新
    user.yaml                       # 用户差量模板
    robots/
      talon/
        README.md
        robot.yaml
        hardware.yaml
        ecm.yaml                    # 型号相关主站配置
        robot.urdf
        eni7.xml
        meshes/
        mjcf/
      rcm/
        README.md
        robot.yaml
        hardware.yaml
        master.yaml
        RCM.urdf
        meshes/
        mjcf/
    devices/
      talon_001/
        README.md                   # 设备信息与资料说明
        device.yaml                 # 本设备配置及所用结果的引用
        identification/
          joint_zero/
          kinematics/
          dynamics/
          payload/
          tool_tcp/
        maintenance/                # 维护记录
      talon_002/
        README.md
        device.yaml
        identification/
```

```
41          maintenance/
42        rcm_001/
43          README.md
44          device.yaml
45          identification/
46          maintenance/
47      tools/
48        demo_gripper/
49        robotiq_2f85/
50      scripts/
51        start_app.sh
52        start_ecm.sh
53        start_igh.sh
```

config 根目录提供通用入口模板，具体使用哪台设备、哪种主站以及哪些现场参数，由 rocos-cfg 中的实际工程配置决定。完成配置后，用导出的文件更新目标目录中的对应入口。robots 下保存型号相关内容，devices 下保存设备专属内容，入口文件引用这些资料。

3 每台机器人建立独立档案

设备目录命名

devices 下采用 **<名称>_<序列号>** 命名，例如 talon_001、talon_002、rcm_001。名称使用统一的型号或设备名称，序列号沿用设备实际编号并保留前导零。同一台设备长期使用同一目录；更换计算机、维护或重新标定时继续在原目录积累资料。

每个目录就是这台机器人的独立文件档案库。首版使用 YAML、NPZ、CSV、说明文件等已有文件保存数据，无需引入数据库服务。型号通用模型放在 robots，针对本设备标定得到的运动学、动力学及其他参数放在自己的 devices 目录中。

需要保留的数据

资料	保存位置	后续用途
设备配置与安装信息	device.yaml、README.md	记录设备身份、所用型号、工具安装和当前引用的标定结果
关节零点	identification/joint_zero/	保存各次零点测量与结果，供本设备恢复配置或维护使用
运动学标定	identification/kinematics/	保存标定参数、标定后的模型、原始测量和说明，供本设备再次部署及后续算法使用

动力学辨识	identification/dynamics/	保存辨识参数、原始采样和说明，供本设备后续控制与辨识复用
负载与 TCP 标定	identification/payload/、tool_tcp/	保留对应工具、安装状态和结果，便于重新使用同一配置
维护记录	maintenance/	记录维修、部件更换和重新标定等情况，说明参数变化原因

保留每次标定结果

新的标定结果按日期或“日期_次数”建立子目录，不覆盖已有结果。README 简要记录标定时间、使用工具、设备安装状态、结果用途及相关文件；device.yaml 或设备说明明确当前使用哪一份结果。使用程序需要哪种格式，就保留对应文件，不在初版统一改造数据格式。

Code block

```
devices/talon_001/
  device.yaml
  identification/
    kinematics/
      2026-11-05_01/
        parameters.yaml          # 参数文件 示意
        calibrated.urdf          # 如本次产生标定模型
        raw/                     # 原始数据
        README.md
    dynamics/
      2026-11-05_01/
        parameters.npz           # 保留辨识工具实际输出
        raw/
        README.md
```

复用时按设备名称和序列号找到原档案，再选用适合当前安装状态的结果。运动学和动力学文件先完整保存；程序是否能直接应用这些文件，由对应程序的实际功能决定，保存资料不等于已接入运行加载。

4 统一配置目录与启动约定

统一放到指定路径下的 config 目录

部署时先将仓库中的 config 目录放到 <指定路径>/config，例如 /opt/rocos/config，再根据 rocos-cfg 的实际工程配置更新其中的入口模板及相关配置。各程序共同使用配置后的这个目录，模型、设备标定、工具与脚本都从这里取用。

```
/opt/rocos/
  config/
    app.yaml
    ecm.yaml
    igh.yaml
    user.yaml
    robots/
    devices/
    tools/
    scripts/
```

各程序使用自己的启动入口

目标约定：rocos-app、rocos-ecm、rocos-igh 等都通过 --config 指定同一个目录；程序在该目录读取经 rocos-cfg 配置更新后的启动文件，相关启动脚本统一保存在 scripts/。例如 App 对应 app.yaml，ECM 对应 ecm.yaml，IgH 对应 igh.yaml。

Code block

```
# 以下为计划采用的目录参数用法
rocosAppMain --config /opt/rocos/config
rocos_ecm --config /opt/rocos/config
rocos_igh_master --config /opt/rocos/config
```

当前实现说明：现有程序的 --config 接收具体 YAML 文件。统一传目录需要后续为各程序补充入口选择逻辑；实现前可分别指定 /opt/rocos/config/app.yaml、ecm.yaml、igh.yaml。目录设计和资料集中整理可以先完成。

启动脚本负责将经 rocos-cfg 配置更新后的入口交给相应程序；本次使用的型号、设备和主站由工程配置确定。App 与实际选用的主站引用同一台设备和同一型号资料。相对资源路径沿用声明它的配置文件所在目录作为起点，移动整个 config 目录时保留其内部结构。

5 下载与日常维护

获取资料

初版通过 GitHub 仓库克隆或 Download ZIP 下载资料。使用流程为：下载资源 → 在 rocos-cfg 中基于模板配置实际工程 → 更新或导出入口文件及相关配置 → 放入目标 config 目录 → 启动程序。仅需一个型号或一台设备时，可提供对应目录的普通压缩包，同时带上入口模板及其引用的型号、工具等资料，解压后保持相同目录结构。

日常更新

新增型号时在 robots 下增加对应目录及说明；新增设备时在 devices 下建立“名称_序列号”目录；完成标定后，将结果和原始数据保存到该设备目录，并更新当前使用结果的说明或引用。通用工具资料继续放在 tools 下。

以 rocos-resources 作为资料的统一维护位置。调整设备选择、主站或现场参数时，通过 rocos-cfg 更新工程配置，再更新目标目录中的对应文件；更新入口模板或导出工程时，保留其他设备档案和已有标定历史。现场产生的新参数、标定和维护记录整理后提交回原设备目录，利用 Git 提交记录保留变更历史。

6 初版整理步骤

1. 建立 rocos-resources 仓库和 config 目录，将现有 robots、devices、tools 及相关脚本集中整理进去。
2. 先整理 Talon、RCM 等已有型号，保留现有模型文件、硬件配置、ENI 和仿真资料及其相对路径。
3. 按“名称_序列号”为已有设备建立目录，将各自运动学、动力学、零点、负载与 TCP 等结果归档到对应设备下。
4. 整理 App、ECM、IgH 等程序的入口模板和启动脚本，说明如何通过 rocos-cfg 选择型号、设备与主站，并更新或导出对应配置。
5. 将整理后的 config 目录放到 /opt/rocos/config 等指定位置，使用 rocos-cfg 的实际工程配置更新入口模板；先按现有文件参数使用，后续补齐 --config 目录参数的支持。
6. 在仓库 README 中说明目录、下载、rocos-cfg 配置与入口更新、启动、新增设备和保存标定结果的方法，后续沿用同一结构维护。

初版交付是一份结构清楚、资料完整、可以下载的资源仓库，包含各程序的入口模板，以及通过 rocos-cfg 配置更新后共用 config 目录的使用说明。设备专属标定资料持续保存在同一设备档案中，供以后维护和复用。